

EXECUTIVE BRIEF

AI-assisted **source-to-target** mapping for data conversions.

AI Mapping Studio turns the slowest, most error-prone part of an enterprise data conversion — deciding how every target column is populated from a legacy source — into an AI-generated draft that analysts review, correct, validate, and ship as runnable SQL. Insurance / Guidewire-inspired, built for data conversion teams.

8

AI-assisted features

Multi

Tenant & data-isolated

34

API endpoints

SQL + AI

Live DB & Claude

0

Build steps to run

— THE PROBLEM

A conversion lives or dies on its mapping.

Every conversion needs a column-level Source-to-Target Mapping: for each target field, which source column feeds it, what transformation applies, and how keys and joins line up. Across hundreds of tables this is slow, manual, and easy to get wrong. AI Mapping Studio proposes the mapping, then keeps a human firmly in control of approving it.

INGEST

Bring in any schema

Connect a live SQL Server, or upload a data dictionary, DDL, or spreadsheet — extracted automatically, with instant deterministic parsing for SQL and structured Excel.

GENERATE

AI drafts the mapping

Claude proposes each column's source, transformation rule, business rule, null handling, a 0–100 confidence, and the SQL JOIN that assembles each target entity.

SHIP

Review → validate → SQL

Approve, edit, or regenerate in the workspace; run validation rules; generate ETL stored procedures and deploy to SQL Server with an AI fix step that a person signs off.

The workflow, end to end.

Source & Target systems

Manage connections — live SQL Server or uploaded files — with exactly one active target schema driving the whole app.

AI Mapping Generator

Pick target tables, tune the strategy and the editable prompt, and generate mappings table-by-table with a live console.

Mapping Workspace

Confidence scoring, approve/reject, inline edits, single-field regeneration, and a full audit trail of every change.

Validation

A rules engine flags low-confidence and unmapped-mandatory fields against configurable thresholds.

ETL & Deploy

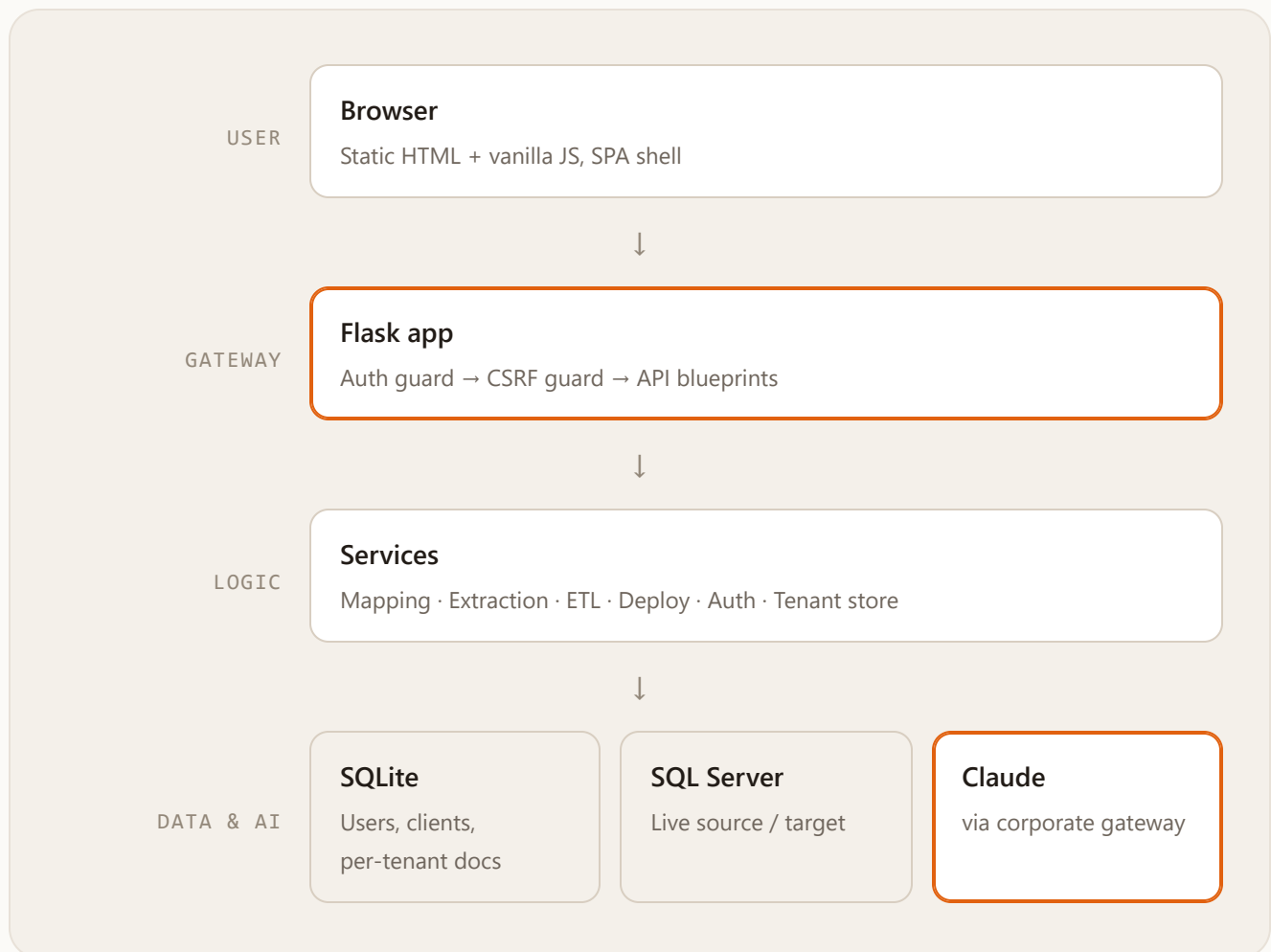
Generate stored procedures and CREATE TABLE DDL, edit them, and deploy with all-or-nothing transactions.

Export & Usage

Export the mapping document (CSV/JSON) and track AI token usage per feature and per client.

One service, cleanly layered.

A static browser frontend and the API are served from a single origin. Everything privileged — database access, AI calls, persistence — happens on the server, scoped to the signed-in user and their active client.



Frontend: HTML + vanilla JavaScript, no framework, no build step. Backend: Flask, layered **api** → **services** → **parsers/schemas** → **core**. Two SQLite files hold the app's own data; live SQL Server is only the conversion's source and target.

Claude, applied narrowly and safely.

Eight bounded, single-request features — each returns structured data or SQL that the application parses and a person reviews. The model never acts on its own.

- | | |
|---|---|
| • Mapping generation — per target table | • Single-field regeneration — grounded on the real source |
| • Schema extraction — from uploaded files | • Rich extraction — infers keys & nullability |
| • ETL stored-procedure generation | • CREATE TABLE DDL generation |
| • Add column / entity from plain language | • Deploy-time SQL auto-fix — human-gated |

Guardrails. Prompts forbid inventing tables or columns; hallucinated DDL is flagged; a deploy that fails is auto-corrected but **never auto-deployed** — the fix is surfaced for review. Usage logs record token counts only — never prompt or response content. **No RAG, no vector database, and no autonomous agents.**

What's in place — and what's next.

An honest posture: strong tenant isolation and API hardening today; production-hardening still ahead. Nothing below is claimed unless it's in the code.

In place

Implemented

- ✓ Email/password auth, hashed passwords, admin-managed accounts (self-signup off by default)
- ✓ Per-tenant data isolation scoped to user + client, enforced server-side
- ✓ CSRF protection, login throttling, and connection-attempt rate limiting
- ✓ Connection passwords never persisted; no credentials in logs or AI prompts
- ✓ Human-gated deploys; all-or-nothing transactions with rollback

Recommended next

Roadmap

- Run behind a production web server with debug off and HTTPS (currently a dev server)
- Add CI, containerization, and automated frontend tests
- Enforce upload size limits and pin dependency versions
- Move to Postgres & shared storage if concurrency grows (schema is already portable)

— WHERE IT STANDS

Working software, documented.

● Core workflow functional

● Backend test suite (30 files)

● Full technical docs in `docs/`

● Not yet production-hardened

The codebase is clean and layered, with the security-critical paths (auth, tenant isolation, CSRF, deploy) covered by tests. The remaining work is concentrated in deployment readiness and frontend test coverage — not in core correctness.

Prepared from a full source-code analysis of AI Mapping Studio. The complete 28-document technical reference — architecture, API, prompt inventory, database, security, testing, and a developer guide — lives in the repository under `docs/`, indexed by `docs/00-README.md`.

This brief summarizes an internal engineering documentation set. Figures (features, endpoints, tests) reflect the code as analyzed. No secrets or credentials are included.